Лекция 3. Первая модель: предсказываем цену квартиры

На прошлом занятии мы подготовили таблицу с квартирами. Теперь попробуем по характеристикам квартиры предсказать её цену.

Для этого используем линейную регрессию. Сначала разберём, как она получает прогноз и как измерить ошибку. Затем повторим шаги из `Lecture_3_light.ipynb`: загрузим данные, обучим модель, найдём утечку и сравним результаты.

1. Что мы хотим предсказать

Представим таблицу:

Квартира	Площадь, м²	Комнаты	Цена, руб.
А	30	1	3 000 000
Б	50	2	5 000 000
В	70	3	7 000 000
Новая квартира	60	2	?

Площадь и количество комнат известны. Цену новой квартиры хотим получить с помощью модели.

Признаки описывают объект. В нашем случае это площадь, комнатность, этаж, город и другие характеристики квартиры.

Целевая переменная, или ответ, содержит то, что мы предсказываем. В ноутбуке это столбец `price`, цена в рублях.

Задача предсказания числа называется **регрессией**. Цена квартиры, время доставки и расход топлива являются примерами таких ответов.

Для обучения нужны строки, в которых ответ уже известен. Модель использует эти примеры, чтобы подобрать общее правило. Потом это правило можно применить к другой квартире.

Таблица выше учебная. В реальных данных квартиры с одинаковой площадью могут стоить по-разному, поэтому точного совпадения всех прогнозов с ценами обычно не получается.

2. Как работает линейная регрессия

Один признак

Начнём только с площади. Линейная регрессия использует правило:

$$\hat{y} = b + wx.$$

Обозначения:

Символ	Значение
x	Известный признак, например площадь
w	Вес: число, на которое умножаем признак
b	Свободный член: постоянная добавка
$\hat{y}$	Прогноз модели, читается «игрек с крышкой»
y	Настоящий ответ из таблицы

Допустим, модель получила такое правило:

$$\text{прогноз цены} = 500\,000 + 80\,000 \cdot \text{площадь}.$$

Для квартиры площадью 50 м²:

$$\hat{y} = 500\,000 + 80\,000 \cdot 50 = 4\,500\,000 \text{ руб.}$$

В этой формуле каждый дополнительный квадратный метр увеличивает прогноз на 80 000 рублей. Свободный член добавляет одинаковые 500 000 рублей к каждому прогнозу.

Зачем нужна постоянная добавка? Без неё правило $\hat{y} = wx$ всегда даёт ноль при $x = 0$. Свободный член позволяет поднимать или опускать всю линию, сохраняя её наклон. При этом прогноз для площади 0 м² не нужно понимать как цену настоящей квартиры: это лишь результат подстановки в формулу.

Несколько признаков

В ноутбуке признаков много. Для двух признаков правило выглядит так:

$$\hat{y} = b + w_1 x_1 + w_2 x_2.$$

Например:

$$\text{прогноз цены} = 500\,000 + 70\,000 \cdot \text{площадь} + 300\,000 \cdot \text{комнаты}.$$

Для 50 м² и двух комнат прогноз составит:

$$500\,000 + 70\,000 \cdot 50 + 300\,000 \cdot 2 = 4\,600\,000 \text{ руб.}$$

У каждого признака свой вес. Модель умножает значения признаков на веса, складывает результаты и добавляет свободный член.

Все коэффициенты в этих примерах придуманы для объяснения. В ноутбуке их подбирает алгоритм по данным.

Что значит обучить модель

Обучить линейную регрессию означает подобрать веса и свободный член, чтобы уменьшить ошибки на обучающих примерах.

Площади и настоящие цены уже записаны в таблице. Во время обучения меняются параметры формулы, а не исходные значения квартир.

Для обычной линейной регрессии выбираются параметры, при которых сумма квадратов ошибок на обучающих данных минимальна. Этот подход называется **методом наименьших квадратов**. Как считать такую ошибку, разберём ниже.

3. Подключаем библиотеки

В ноутбуке используются следующие инструменты:

```
from pathlib import Path

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", "{:,.2f}".format)
```

`Path` помогает задать путь к файлу. `pandas` читает данные и работает с таблицами. `numpy` понадобится для квадратного корня. `matplotlib` строит графики.

Scikit-learn, в коде `sklearn`, содержит готовые инструменты машинного обучения. Мы берём из неё разделение данных, линейную регрессию и расчёт ошибки.

Последние две строки меняют только отображение таблиц: разрешают показывать до 30 столбцов и выводят числа с двумя знаками после запятой. Сами данные от этого не округляются.

4. Загружаем и проверяем таблицу

Используем файл `data_model_final.csv`, подготовленный на прошлом занятии.

```
data_path = Path("../занятие_2/data_model_final.csv")

if not data_path.exists():
    data_path = Path("занятие_2/data_model_final.csv")

df = pd.read_csv(data_path)

print("Строк и столбцов:", df.shape)
df.head()
```

Первый путь рассчитан на запуск из папки `занятие_3`. Символы `..` означают переход на одну папку выше. Второй путь рассчитан на запуск из корня курса. Файл должен находиться по одному из этих путей; иначе нужно указать его фактическое расположение.

`pd.read_csv()` читает CSV-файл в таблицу `df`. Одна строка соответствует одной квартире, один столбец содержит одну характеристику.

`df.shape` возвращает два числа: количество строк и количество столбцов. `df.head()` показывает первые пять строк и помогает убедиться, что загрузилась нужная таблица.

Типы и пропуски

```
df.info()
print("Всего пропущенных значений:", df.isna().sum().sum())
```

`info()` показывает названия столбцов, типы данных и количество заполненных значений. `int64` обозначает целые числа, `float64` обозначает числа с дробной частью.

В выражении `df.isna().sum().sum()` сначала отмечаются пропуски, затем считается их количество в каждом столбце, потом складываются результаты по всей таблице. Ноль означает, что пропусков нет.

Основные статистики

```
df[["Общая площадь", "rooms", "price", "price_per_m2_calc"]].describe()
```

Двойные квадратные скобки выбирают несколько столбцов. Метод `describe()` показывает их основные характеристики:

Строка результата	Что она показывает
`count`	Количество заполненных значений
`mean`	Среднее арифметическое
`std`	Стандартное отклонение, характеристику разброса
`min`	Минимальное значение
`25%`	Первый квартиль: примерно четверть значений не выше него
`50%`	Медиану, середину отсортированных значений
`75%`	Третий квартиль: примерно три четверти значений не выше него
`max`	Максимальное значение

Для чисел `30, 40, 50` среднее равно $(30 + 40 + 50)/3 = 40$, медиана тоже равна 40. Если заменить 50 на 500, среднее станет 190, а медиана останется 40. Поэтому очень дорогие квартиры могут заметно поднять среднюю цену.

Стандартное отклонение помогает оценить, насколько сильно значения различаются. У наборов `39, 40, 41` и `10, 40, 70` одинаковое среднее, но у второго разброс больше.

Для расчёта разброса находят отклонения от среднего, возводят их в квадрат и усредняют. Получается **дисперсия**. Квадратный корень из неё даёт стандартное отклонение в исходных единицах. В `pandas` при расчёте `std` по умолчанию сумму квадратов делят на $n - 1$, где n обозначает число заполненных значений.

Проверка таблицы нужна до обучения. Она помогает заметить пропуски, неожиданные типы и подозрительные значения.

5. Отделяем признаки от ответа

```
X = df.drop(columns=["price"])
y = df["price"]
```

`X` содержит таблицу признаков. `y` содержит столбец настоящих цен.

Метод `drop(columns=["price"])` создаёт таблицу без столбца цены. Исходная таблица `df` при таком вызове сохраняется.

Цену нельзя оставить среди признаков: при настоящем прогнозе она неизвестна. Именно её мы хотим получить.

В подготовленной таблице категории уже представлены числами. Например, столбец `city_Махачкала` содержит 1 для квартиры в Махачкале и 0 для остальных. Такой способ называется **One-Hot Encoding**: вместо названия категории используются отдельные столбцы с нулями и единицами.

Однако числовой столбец ещё не обязательно подходит для прогноза. Важно понять, откуда он получен. Пока в `X` остаётся `price_per_m2_calc`. Позже увидим, почему это проблема.

6. Делим данные на train и test

Нам нужны две части данных:

Часть	Для чего используется
`train`, обучающая выборка	По этим квартирам модель подбирает параметры
`test`, тестовая выборка	На этих квартирах проверяем готовую модель

Если проверять качество только на обучении, результат может быть слишком хорошим: модель уже использовала эти примеры при подборе весов. Нам важно понять, как она работает на других квартирах.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

`test_size=0.2` выделяет около 20% строк для проверки. Около 80% остаётся для обучения.

Признаки и ответы делятся вместе. Если квартира попала в `X_test`, её настоящая цена окажется в `y_test`. Соответствие строк сохраняется.

`random_state=42` фиксирует случайное разделение: с теми же данными и настройками получим те же группы. Число 42 не имеет особого смысла, вместо него можно выбрать другое целое число.

В ноутбуке также есть переменная `SEED = 42`. Её можно передать как `random_state=SEED`. Сама по себе запись `SEED = 42` не влияет на разделение, пока переменная не используется в вызове.

Мы делим строки один раз. Обе модели будут проверяться на одинаковых квартирах, чтобы сравнение было понятным.

7. Обучаем первую модель и получаем прогнозы

```
model_leak = LinearRegression()
model_leak.fit(X_train, y_train)
pred_leak = model_leak.predict(X_test)
```

Здесь три разных действия:

1. `LinearRegression()` создаёт модель. Она ещё не обучена.
2. `fit(X_train, y_train)` подбирает веса и свободный член по признакам и известным ценам обучающих квартир.
3. `predict(X_test)` подставляет признаки тестовых квартир в полученную формулу и возвращает прогнозы.

В `predict()` не передаётся `y_test`. Настоящие тестовые цены понадобятся позже, когда будем сравнивать их с прогнозами.

Модель получает один общий набор весов для всех квартир. Она не создаёт отдельную формулу под каждую строку.

В `LinearRegression` свободный член добавляется по умолчанию, поэтому вручную создавать столбец единиц не нужно.

8. Как измерить ошибку: MSE и RMSE

Ошибка одной квартиры

Сравним настоящую цену с прогнозом:

$$\text{ошибка} = y - \hat{y}.$$

Если квартира стоит 5 млн рублей, а модель предсказала 4 млн, ошибка равна 1 млн рублей. Модель занизила цену.

Если прогноз равен 6 млн, ошибка равна минус 1 млн рублей. Модель завысила цену.

Почему нельзя просто усреднить эти ошибки

Возьмём две квартиры. Для удобства все значения в таблице записаны в миллионах рублей.

Квартира	Настоящая цена	Прогноз	Ошибка	Квадрат ошибки
А	5	4	1	1
Б	5	6	-1	1

Среднее ошибок равно $(1 + (-1))/2 = 0$. Но оба прогноза неверны: положительная и отрицательная ошибки просто погасили друг друга.

Поэтому возведём ошибки в квадрат и найдём среднее:

$$\mathrm{MSE} = \frac{1^2 + (-1)^2}{2} = 1.$$

Это **MSE**, средний квадрат ошибки. В данном расчёте единицы MSE являются квадратами миллионов рублей, поскольку исходные цены записаны в миллионах.

Общая формула:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

n обозначает количество проверяемых квартир, i обозначает номер квартиры, а $\sum$ означает сложение. Формула предлагает сделать четыре действия:

1. Для каждой квартиры вычесть прогноз из настоящей цены.
2. Возвести каждую разность в квадрат.
3. Сложить квадраты.
4. Разделить сумму на количество квартир.

Квадрат убирает знак и усиливает влияние больших промахов. Ошибка 2 даёт квадрат 4, ошибка 4 даёт квадрат 16. Промах увеличился в два раза, а его вклад в MSE увеличился в четыре раза.

Зачем извлекать корень

Если цены записаны в рублях, MSE измеряется в рублях в квадрате. Такие числа трудно связать с обычной стоимостью квартиры.

Извлечём квадратный корень:

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}.$$

RMSE измеряется в тех же единицах, что и цена. В нашем маленьком примере RMSE равна 1 млн рублей. Если считать на исходных ценах в рублях, MSE составит 1 000 000 000 000 руб², а RMSE составит 1 000 000 рублей.

RMSE показывает масштаб ошибок с усиленным влиянием крупных промахов. Это не обычное среднее абсолютных ошибок и не обещание, что каждая квартира оценена с такой погрешностью.

Расчёт в ноутбуке

```
mse_leak = mean_squared_error(y_test, pred_leak)
rmse_leak = np.sqrt(mse_leak)

print(f"MSE с утечкой: {mse_leak:,.2f} руб²")
print(f"RMSE с утечкой: {rmse_leak:,.2f} руб.")
```

`mean_squared_error()` сравнивает настоящие цены и прогнозы. `np.sqrt()` извлекает квадратный корень.

При сравнении на одинаковой тестовой выборке меньшие MSE и RMSE означают меньшую ошибку. Ноль означает точное совпадение всех прогнозов с ответами на этой выборке.

На обучении квадратичная ошибка служит целью подбора параметров, то есть **функцией потерь**. На тесте она служит **метрикой качества**, которая оценивает готовые прогнозы. Вычисление похоже, но назначение и данные разные.

9. Почему первая модель использует утечку

В признаках остался столбец `price_per_m2_calc`. Он рассчитан из цены:

$$\text{цена за м}^2 = \frac{\text{price}}{\text{площадь}}.$$

Например, квартира стоит 5 000 000 рублей и имеет площадь 50 м². Тогда цена за метр равна 100 000 рублей.

Но если цена новой квартиры неизвестна, вычислить этот признак таким способом нельзя. Для его расчёта сначала нужен тот самый ответ, который мы собираемся предсказывать.

Утечка целевой переменной возникает, когда модель получает информацию об ответе, которая будет недоступна при настоящем прогнозе.

Удаления `price` оказалось недостаточно: часть информации о цене сохранилась в производном столбце. Разделение на train и test тоже не исправляет эту проблему. Даже в тестовых строках признак уже рассчитан из настоящих цен.

Может возникнуть вопрос: почему ошибка тогда не равна нулю? Ведь цену можно восстановить умножением площади на цену за метр.

Линейная регрессия в этом примере складывает взвешенные признаки:

$$\hat{y} = b + w_1 \cdot \text{площадь} + w_2 \cdot \text{цена за м}^2 + \dots$$

Она не перемножает эти два столбца между собой. Поэтому утечка может улучшить результат, но не обязана сделать его идеальным.

Перед использованием любого признака полезно задать вопрос: «Будет ли это значение известно в тот момент, когда понадобится прогноз?» Для `price_per_m2_calc` в нашей задаче ответ отрицательный.

10. Удаляем утечку и обучаем модель заново

```
X_train_clean = X_train.drop(columns=["price_per_m2_calc"])
X_test_clean = X_test.drop(columns=["price_per_m2_calc"])

model_clean = LinearRegression()
model_clean.fit(X_train_clean, y_train)

pred_clean = model_clean.predict(X_test_clean)
```

Удаляем столбец и из обучающих, и из тестовых признаков. Для обучения и прогноза модели нужен одинаковый набор столбцов в одинаковом порядке.

Квартиры остаются теми же. `y_train` и `y_test` тоже сохраняются: правильные ответы не изменились.

Модель нужно обучить заново. Первая модель подбирала веса с учётом цены за метр. После удаления признака требуется подобрать параметры для оставшейся информации.

Теперь прогноз строится по площади, комнатности, этажу, городу и другим оставшимся характеристикам.

Считаем ошибки новой модели:

```
mse_clean = mean_squared_error(y_test, pred_clean)
rmse_clean = np.sqrt(mse_clean)

print(f"MSE без утечки цены за м²: {mse_clean:,.2f} руб²")
print(f"RMSE без утечки цены за м²: {rmse_clean:,.2f} руб.")
```

Увеличение ошибки после удаления утечки не означает, что столбец нужно вернуть. Проверка стала ближе к реальной задаче, в которой цена заранее неизвестна.

11. Сравниваем результаты

Общая ошибка двух моделей

```
results = pd.DataFrame({
    "Модель": ["С утечкой", "Без цены за м²"],
    "MSE, руб²": [mse_leak, mse_clean],
    "RMSE, руб.": [rmse_leak, rmse_clean]
})

results
```

В каждой строке таблицы одна модель, в столбцах её ошибки. Обе модели проверены на одних квартирах. Сравниваем результат при изменении набора признаков, а не разных тестовых выборок.

В сохранённых выводах вычислительных ячеек `Lecture_3_light.ipynb` получились такие значения:

Модель	RMSE, примерно
С утечкой	707 тыс. руб.
Без цены за м²	1,48 млн руб.

Текст заключения в ноутбуке содержит другие числа: 1,41 и 2,06 млн рублей. Они не совпадают с сохранёнными выводами ячеек. При самостоятельном запуске ориентируемся на заново рассчитанную таблицу `results`: после изменения данных результаты могут отличаться.

Первое значение меньше, но получено с использованием информации о правильном ответе. Второе отражает результат после удаления этой утечки.

Нельзя по одной величине RMSE решить, подходит ли модель для любой задачи. Нужно учитывать стоимость квартир и допустимую ошибку. Промах в миллион рублей имеет разное значение для квартиры за 2 млн и за 30 млн.

Прогнозы отдельных квартир

```
comparison = pd.DataFrame({
    "Настоящая цена": y_test,
    "Прогноз с утечкой": pred_leak,
    "Прогноз без цены за м²": pred_clean
})

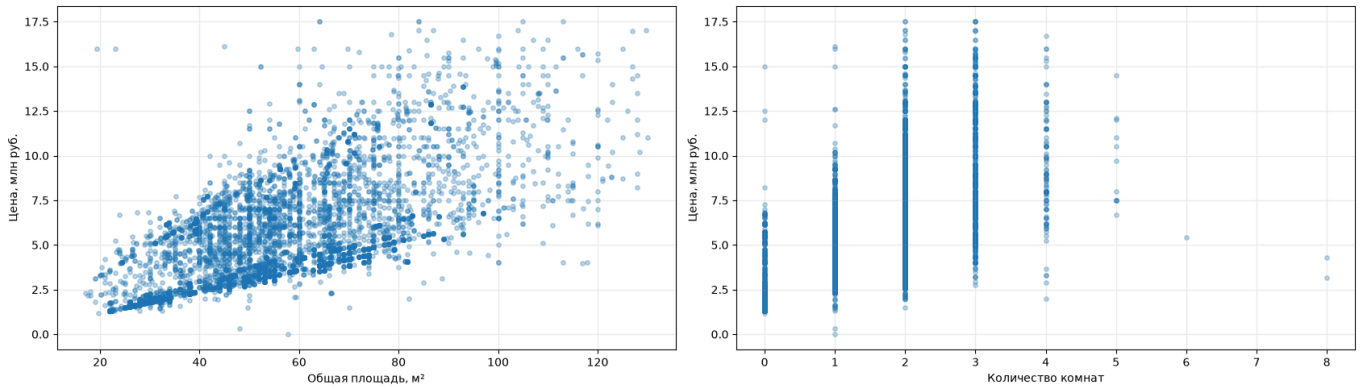
comparison.head(10)
```

Одна строка показывает настоящую цену квартиры и два прогноза для неё. Так можно увидеть, где модель завысила цену, а где занизила.

Общая метрика объединяет ошибки всех тестовых квартир в одно число. Первые десять строк дают конкретные примеры, но не заменяют оценку на всей тестовой выборке.

12. Читаем графики

В конце ноутбука строятся две диаграммы рассеяния. На первой сравниваются площадь и цена, на второй количество комнат и цена.




```
features = ["Общая площадь", "rooms"]
titles = ["Общая площадь, м²", "Количество комнат"]

fig, axes = plt.subplots(1, 2, figsize=(17, 5))

for ax, feature, title in zip(axes, features, titles):
    ax.scatter(df[feature], df["price"] / 1_000_000, alpha=0.3, s=15)
    ax.set_xlabel(title)
    ax.set_ylabel("Цена, млн руб.")
    ax.ticklabel_format(axis="x", style="plain", useOffset=False)
    ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()
```

Каждая точка представляет одну квартиру. По горизонтали находится значение признака, по вертикали настоящая цена в миллионах рублей. Деление на `1_000_000` сделано для удобства отображения и не меняет цены, использованные при обучении.

`plt.subplots(1, 2)` создаёт два графика в одном ряду. Цикл `for` рисует их по очереди. `alpha=0.3` делает точки полупрозрачными, чтобы было легче увидеть их скопления.

На графике площади смотрим, растёт ли цена вместе с площадью и насколько сильно отличаются цены похожих по размеру квартир. Чем шире разброс цен при одной площади, тем труднее точно предсказать цену только по ней.

На графике комнат точки собираются в вертикальные полосы, потому что количество комнат принимает отдельные целые значения: 1, 2, 3 и так далее. В одной полосе могут быть квартиры с разными площадями, расположением и состоянием, поэтому цены различаются.

Эти графики показывают связь признаков с настоящей ценой во всей таблице. Они не показывают ошибки модели: для оценки прогнозов мы уже использовали `y_test`, MSE и RMSE.